Práctica 2: Pruebas de carga, Locust y Prometheus

Evaluación de Sistemas Informáticos

Grupo 16

18 de mayo de 2026

Índice

1. Descripción general del entorno
2. ¿Cómo hemos trabajado?
3. Modificaciones en el código para la práctica
4. Enfoque del monitoreo
5. Pruebas de Carga con Locust
6. Grafana
   - Login
     - Fallos en los *logins* de usuarios
     - Fallos en los *logins* de usuarios
     - Fallos de autenticación por usuarios no validos
     - Flujos de autenticación válidos
     - Flujos de autenticación válidos
     - Flujos de autenticación válidos
     - Peticiones
     - Ratio Peticiones
     - Peticiones a los *endpoints*
     - Respuestas a peticiones
     - Tiempo de respuesta a los diferentes *endpoints*
   - *Tokens*
     - Ratio de *Tokens* en circulación en nuestro sistema
     - Ratio de *Tokens* en circulación en nuestro sistema

¿Qué elementos son clave?

- Servidor de autenticación OAuth2 desarrollado en Python (Flask y SQLite)
- Implementa flujos clave del RFC 6749: *Authorization Code*, *Client Credentials* y *Refresh Token*.
- Ejecutado mediante el gestor `uv` en el puerto 5001.

Distribución del trabajo

Marcos	33 %	Desarrollo de scripts del fichero Locust y redacción del cuaderno de bitácora
Edgar	33 %	Desarrollo de las metricas y de *models.py*,conexión con el Grafana y documentación
Álvaro	33 %	Gestión de la VM y del repositorio de GitHub, modificación de *oauth2_routes*, creación del *test.py* y documentación

Modificaciones en el Backend

La instrumentación se ha integrado siguiendo el principio DRY (intentando evitar los *circular imports*):

- `metrics.py`: Centralización de métricas (Counters, Gauges, Histograms) para evitar importaciones circulares. Uso estratégico de etiquetas (ej. `grant_type`)
- `models.py`: Auditoría de la base de datos (códigos generados/consumidos, detección de tokens revocados).
- `oauth2_routes.py`: Seguimiento del ciclo de vida de los flujos de autenticación (iniciados, exitosos, fallidos).

Arquitectura de Monitorización

Enfoque EDR (*Endpoint Detection and Response*):

- **Rendimiento:** Histogramas para evaluar tiempos de respuesta y control de peticiones simultáneas.
- **Estado de Tokens:** Monitorización de tokens activos, revocados, códigos pendientes y expirados.
- **Seguridad:** Detección de ataques de fuerza bruta (M2M y Web) e intentos de *Token Replay*.

Diseño de Escenarios en Locust

En el `locustfile.py` hemos intentado simular un comportamiento real"**50 % tráfico legítimo, 50 % tráfico malicioso**.

Tarea	Tipo	Peso	Prob.
`valid_client_credentials`	Tráfico legítimo M2M	4	40 %
`brute_force_api`	Ataque fuerza bruta M2M	2	20 %
`brute_force_user_login`	Ataque fuerza bruta web	2	20 %
`valid_user_login`	Tráfico legítimo web	1	10 %
`replay_attack`	Ataque token replay	1	10 %

Ejecución y Verificación

Configuración de Pruebas:

- **Carga normal:** 50 usuarios, *spawn rate* 5/s durante 5 min].
- **Prueba de estrés:** 200 usuarios, *spawn rate* 10/s durante 3 min].

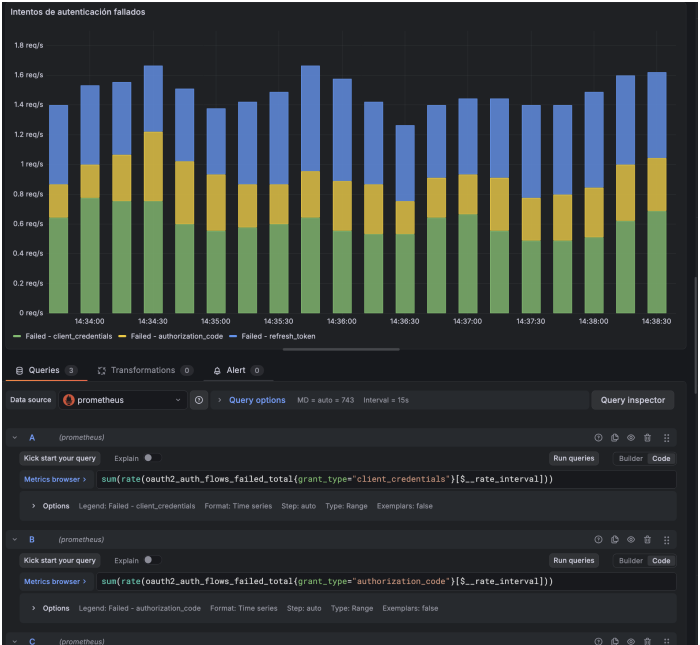
Garantías del sistema:

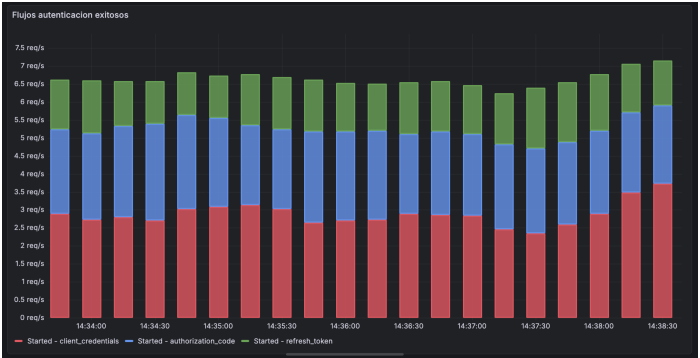
1. Uso de script de verificación (`test_metrics.py`) para validar manualmente los 9 escenarios de métricas antes del ataque global.
2. Limitación de almacenamiento en Prometheus (`retention.size=500MB`) para proteger los recursos de la máquina virtual.


10
Tokens revocados
25.3
Auth Code Reusados
22.7
28.1%
332
143
47.5

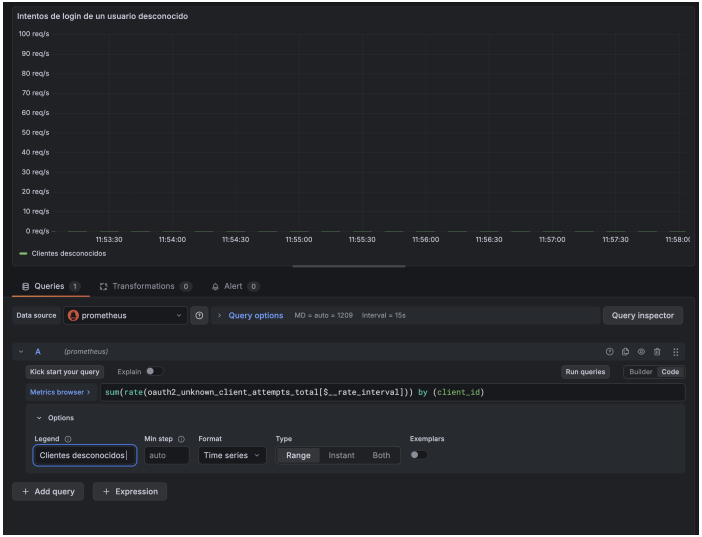

Usuarios con logins erróneos
10
Queries 1
Transformations 0
Data source
prometheus
Query options
MD = auto = 1074
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
oauth2_unique_failed_usernames_active
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression


LoginsHechosCorrectos
2.4
2.2
2
1.8
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
14:31:15
14:31:30
14:31:45
14:32:00
14:32:15
14:32:30
14:32:45
14:33:00
14:33:15
14:33:30
14:33:45
14:34:00
14:34:15
14:34:30
14:34:45
14:35:00
14:35:15
14:35:30
14:35:45
14:36:00
{grant_type="authorization_code", instance="localhost:5001", job="oauth2"}
Queries 2
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 1074
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
rate(oauth2_login_page_visits_total[$__rate_interval])
Options
Legend: {{LoginPageTries}}
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
rate(oauth2_auth_flows_success_total{grant_type="authorization_code"}[$__rate_interval])
Options
Legend: {{LoginsExitosos}}
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression

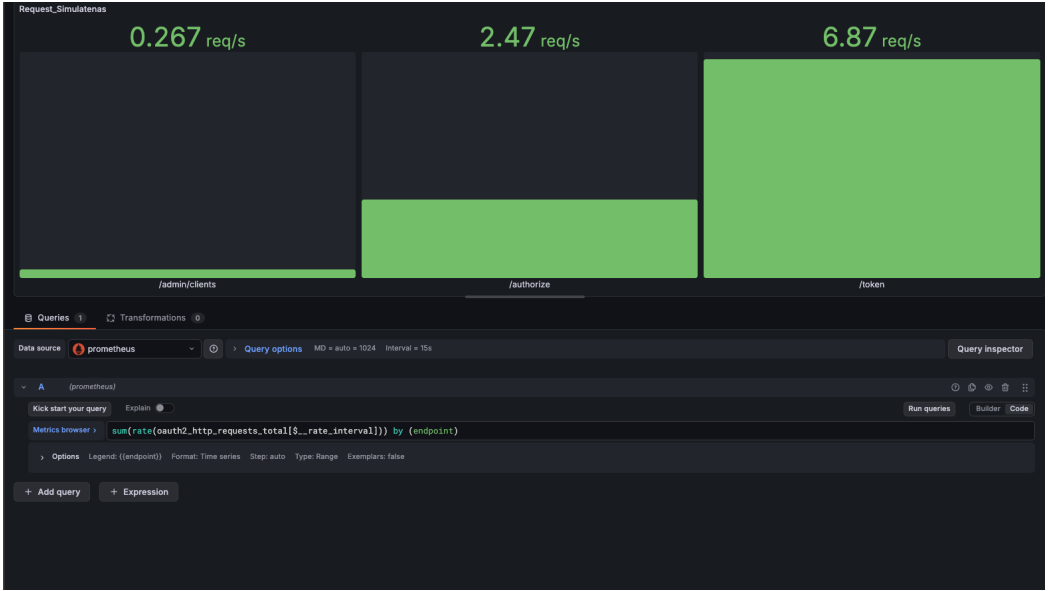

Intentos de autenticación fallados
1.8 req/s
1.6 req/s
1.4 req/s
1.2 req/s
1 req/s
0.8 req/s
0.6 req/s
0.4 req/s
0.2 req/s
0 req/s
14:34:00
14:34:30
14:35:00
14:35:30
14:36:00
14:36:30
14:37:00
14:37:30
14:38:00
14:38:30
Failed - client_credentials
Failed - authorization_code
Failed - refresh_token
Queries 3
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 743
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_auth_flows_failed_total{grant_type="client_credentials"}[$__rate_interval]))
Options
Legend: Failed - client_credentials
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_auth_flows_failed_total{grant_type="authorization_code"}[$__rate_interval]))
Options
Legend: Failed - authorization_code
Format: Time series
Step: auto
Type: Range
Exemplars: false
C
(prometheus)

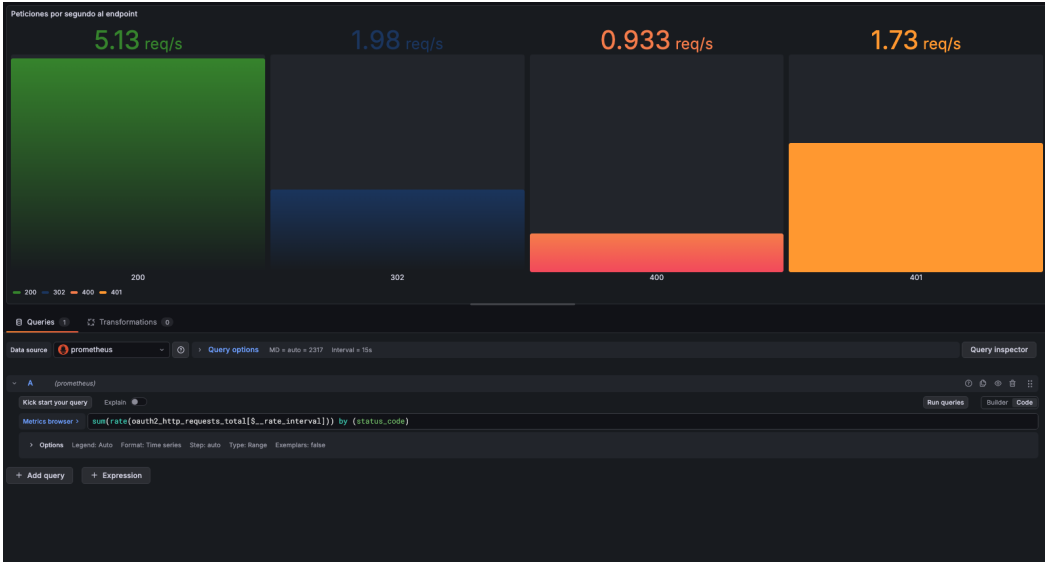

Flujos autenticacion exitosos
7.5 req/s
7 req/s
6.5 req/s
6 req/s
5.5 req/s
5 req/s
4.5 req/s
4 req/s
3.5 req/s
3 req/s
2.5 req/s
2 req/s
1.5 req/s
1 req/s
0.5 req/s
0 req/s
14:34:00
14:34:30
14:35:00
14:35:30
14:36:00
14:36:30
14:37:00
14:37:30
14:38:00
14:38:30
Started - client_credentials
Started - authorization_code
Started - refresh_token
Queries 3
Transformations 0
Alert 0
A (prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_auth_flows_started_total{grant_type="client_credentials"}[$__rate_interval]))
Options
Legend: Started - client_credentials
Format: Time series
Step: auto
Type: Range
Exemplars: false
E (prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_auth_flows_started_total{grant_type="authorization_code"}[$__rate_interval]))
Options
Legend: Started - authorization_code
Format: Time series
Step: auto
Type: Range
Exemplars: false
F (prometheus)
Kick start your query
Explain
Run queries
Builder
Code

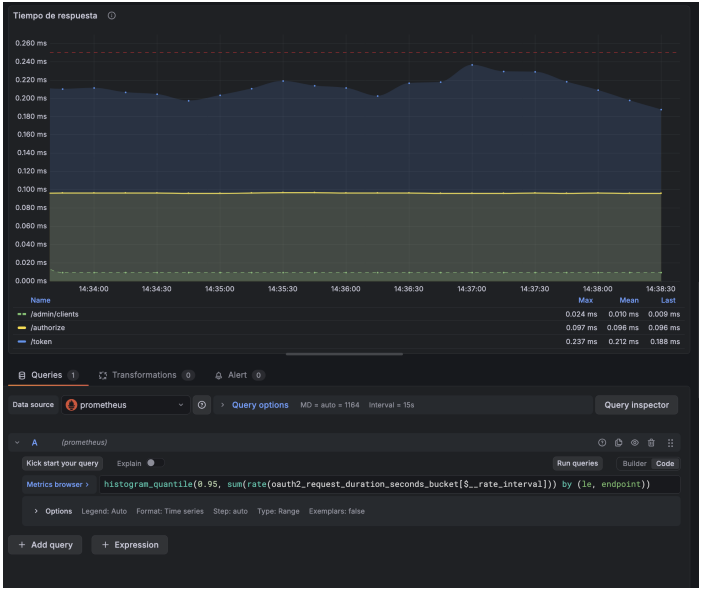

Intentos fallidos de login
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
12:21:00
12:21:15
12:21:30
12:21:45
12:22:00
12:22:15
12:22:30
12:22:45
12:23:00
12:23:15
12:23:30
12:23:45
12:24:00
12:24:15
12:24:30
12:24:45
12:25:00
12:25:15
12:25:30
12:25:45
ana.rodriguez
attacker
carlos.garcia
jane.smith
john.doe
luis.hernandez
maria.lopez
pedro.martinez
sofia.gonzalez
testuser
Queries 1
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 1517
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_user_auth_failed_total[$__rate_interval])) by (username)
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression


Intentos de login de un usuario desconocido
100 req/s
90 req/s
80 req/s
70 req/s
60 req/s
50 req/s
40 req/s
30 req/s
20 req/s
10 req/s
0 req/s
11:53:30
11:54:00
11:54:30
11:55:00
11:55:30
11:56:00
11:56:30
11:57:00
11:57:30
Clientes desconocidos
Queries 1
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 1209
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_unknown_client_attempts_total[$__rate_interval])) by (client_id)
Options
Legend
Min step
Format
Type
Exemplars
Clientes desconocidos
auto
Time series
Range
Instant
Both
Add query
Expression

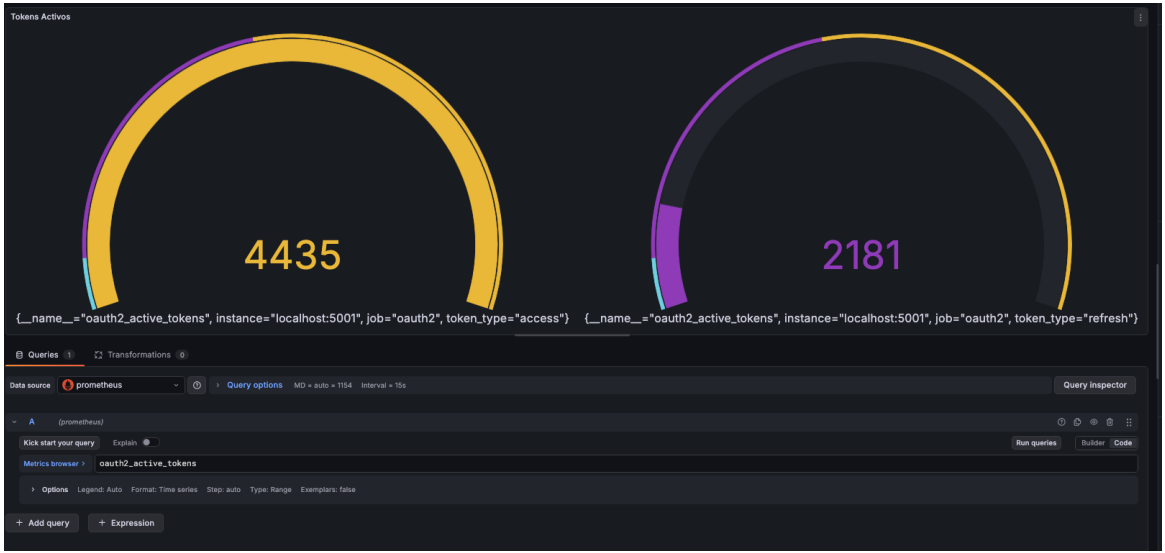

Ratio peticiones exitosas contra fallidas por usuario
28.4%
Queries 1
Transformations 0
Data source
prometheus
Query options
MD = auto = 2056
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_user_auth_failed_total[$__rate_interval])) by (username)
/
sum(rate(oauth2_user_auth_success_total[$__rate_interval])) by (username)
Options
Legend: RatioPeticionesUser
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression

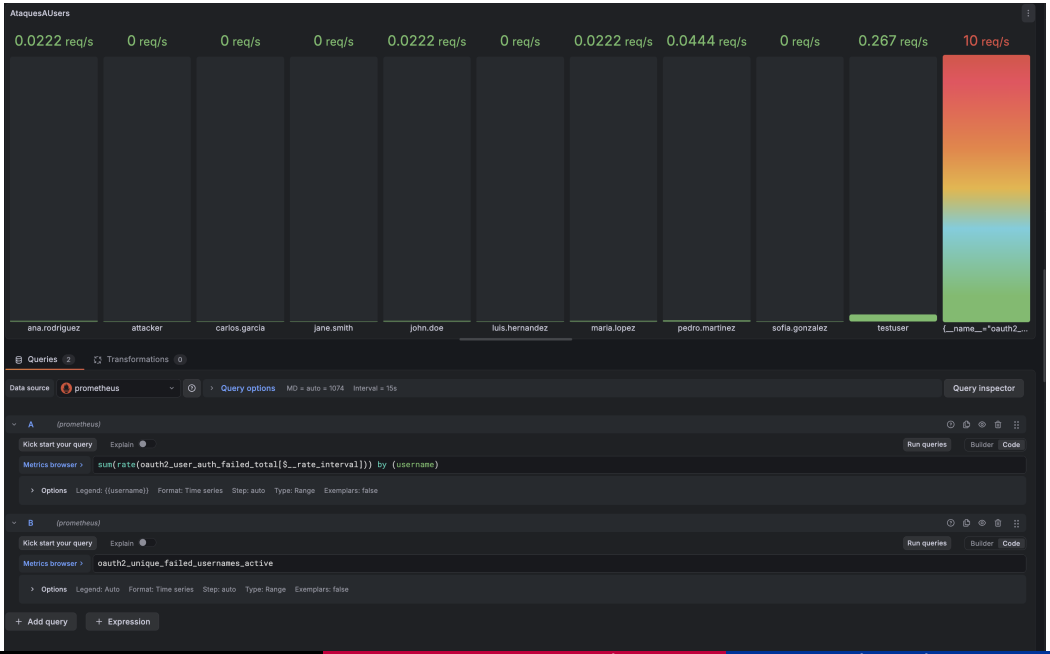

Request_Simulatenas
0.267 req/s
2.47 req/s
6.87 req/s
/admin/clients
/authorize
/token
Queries 1
Transformations 0
Data source
prometheus
Query options
MD = auto = 1024
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_http_requests_total[$__rate_interval])) by (endpoint)
Options
Legend: {{endpoint}}
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression


Peticiones por segundo al endpoint
5.13 req/s
1.98 req/s
0.933 req/s
1.73 req/s
200
302
400
401
200 302 400 401
Queries 1
Transformations 0
Data source
prometheus
Query options
MD = auto = 2317
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_http_requests_total[$__rate_interval])) by (status_code)
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression

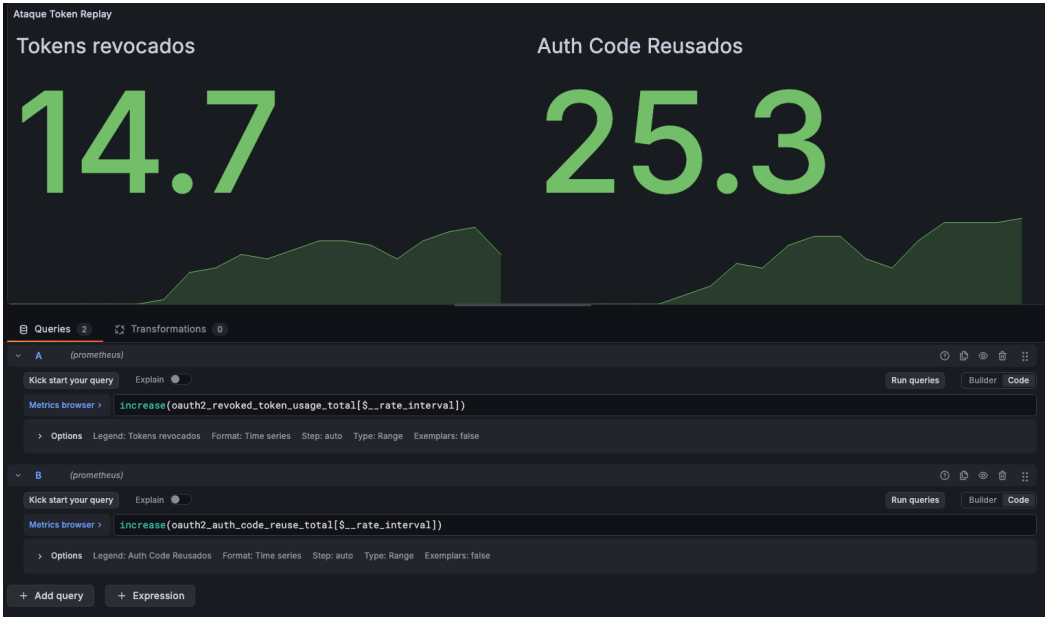

Tiempo de respuesta
0.260 ms
0.240 ms
0.220 ms
0.200 ms
0.180 ms
0.160 ms
0.140 ms
0.120 ms
0.100 ms
0.080 ms
0.060 ms
0.040 ms
0.020 ms
0.000 ms
14:34:00
14:34:30
14:35:00
14:35:30
14:36:00
14:36:30
14:37:00
14:37:30
14:38:00
14:38:30
Name
Max
Mean
Last
/admin/clients
0.024 ms
0.010 ms
0.009 ms
/authorize
0.097 ms
0.096 ms
0.096 ms
/token
0.237 ms
0.212 ms
0.188 ms
Queries 1
Transformations 0
Alert 0
Data source
prometheus
Query options
MD = auto = 1164
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
histogram_quantile(0.95, sum(rate(oauth2_request_duration_seconds_bucket[$__rate_interval])) by (le, endpoint))
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression


Ratio Tokens
4.67 req/s
2.96 req/s
7.62 req/s
Access Tokens/s
Refresh Tokens/s
Total Tokens/s
Queries 3
Transformations 0
A (prometheus)
Kick start your query
Explain
Run queries
Builder Code
Metrics browser
sum(rate(oauth2_access_tokens_issued_total[$__rate_interval]))
Options Legend: Access Tokens/s Format: Time series Step: auto Type: Range Exemplars: false
B (prometheus)
Kick start your query
Explain
Run queries
Builder Code
Metrics browser
sum(rate(oauth2_refresh_tokens_issued_total[$__rate_interval]))
Options Legend: Refresh Tokens/s Format: Time series Step: auto Type: Range Exemplars: false
C (prometheus)
Kick start your query
Explain
Run queries
Builder Code
Metrics browser
sum(rate(oauth2_tokens_issued_total[$__rate_interval]))
Options Legend: Total Tokens/s Format: Time series Step: auto Type: Range Exemplars: false

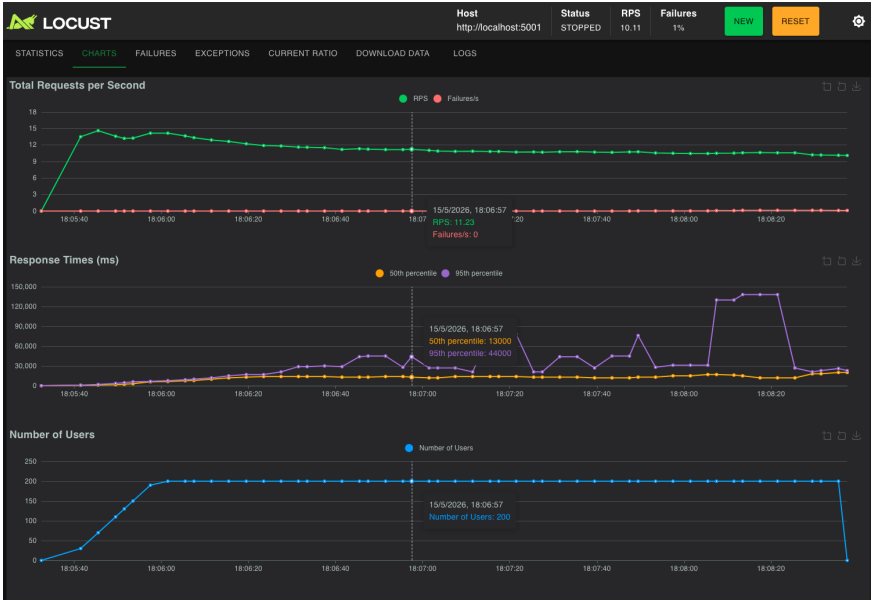

Tokens Activos
4435
2181
{__name__="oauth2_active_tokens", instance="localhost:5001", job="oauth2", token_type="access"}
{__name__="oauth2_active_tokens", instance="localhost:5001", job="oauth2", token_type="refresh"}
Queries 1
Transformations 0
Data source
prometheus
Query options
MD = auto = 1154
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
oauth2_active_tokens
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression

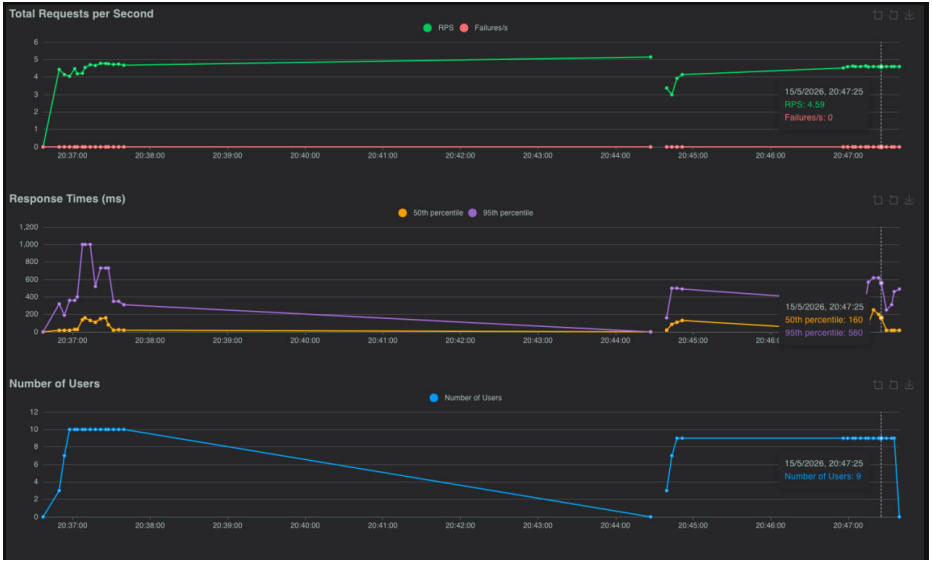

AtaquesAUsers
0.0222 req/s
0 req/s
0 req/s
0 req/s
0.0222 req/s
0 req/s
0.0222 req/s
0.0444 req/s
0 req/s
0.267 req/s
10 req/s
ana.rodriguez
attacker
carlos.garcia
jane.smith
john.doe
luis.hernandez
maria.lopez
pedro.martinez
sofia.gonzalez
testuser
{__name__="oauth2_...
Queries 2
Transformations 0
Data source
prometheus
Query options
MD = auto = 1074
Interval = 15s
Query inspector
A
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
sum(rate(oauth2_user_auth_failed_total[$__rate_interval])) by (username)
Options
Legend: {{username}}
Format: Time series
Step: auto
Type: Range
Exemplars: false
B
(prometheus)
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser
oauth2_unique_failed_usernames_active
Options
Legend: Auto
Format: Time series
Step: auto
Type: Range
Exemplars: false
Add query
Expression


Intentos de Ataque de Fuerza Bruta
0.689 req/s
Queries 1
Transformations 0
Data source
prometheus
Query options
MD = auto = 743
Interval = 15s
Query inspector


Ataque Token Replay
Tokens revocados
14.7
Auth Code Reusados
25.3
Queries 2
Transformations 0
A (prometheus)
Kick start your query
Explain
Run queries
Builder Code
Metrics browser
increase(oauth2_revoked_token_usage_total[$__rate_interval])
Options Legend: Tokens revocados Format: Time series Step: auto Type: Range Exemplars: false
B (prometheus)
Kick start your query
Explain
Run queries
Builder Code
Metrics browser
increase(oauth2_auth_code_reuse_total[$__rate_interval])
Options Legend: Auth Code Reusados Format: Time series Step: auto Type: Range Exemplars: false
+ Add query
+ Expression


LOCUST
Host
http://localhost:5001
Status
STOPPED
RPS
10.11
Failures
0%
NEW
RESET
STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
Total Requests per Second
RPS
Failures/s
Response Times (ms)
50th percentile
95th percentile
Number of Users
Number of Users
18:00:20
18:00:40
18:01:00
18:01:20
18:01:40
18:02:00
18:02:20
18:02:40
18:03:00
18:03:20


LOCUST
Host
http://localhost:5001
Status
STOPPED
RPS
10.11
Failures
1%
NEW
RESET
STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
Total Requests per Second
RPS
Failures/s
18
15
12
9
6
3
0
18:05:40
18:06:00
18:06:20
18:06:40
18:07
18:07:40
18:08:00
18:08:20
15/5/2026, 18:06:57
RPS: 11.23
Failures/s: 0
Response Times (ms)
50th percentile
95th percentile
150,000
120,000
90,000
60,000
30,000
0
18:05:40
18:06:00
18:06:20
18:06:40
18:07:00
18:07:20
18:07:40
18:08:00
18:08:20
15/5/2026, 18:06:57
50th percentile: 13000
95th percentile: 44000
Number of Users
Number of Users
250
200
150
100
50
0
18:05:40
18:06:00
18:06:20
18:06:40
18:07:00
18:07:20
18:07:40
18:08:00
18:08:20
15/5/2026, 18:06:57
Number of Users: 200


Total Requests per Second
RPS
Failures/s
6
5
4
3
2
1
0
20:37:00
20:38:00
20:39:00
20:40:00
20:41:00
20:42:00
20:43:00
20:44:00
20:45:00
20:46:00
20:47:00
15/5/2026, 20:47:25
RPS: 4.59
Failures/s: 0
Response Times (ms)
50th percentile
95th percentile
1,200
1,000
800
600
400
200
0
20:37:00
20:38:00
20:39:00
20:40:00
20:41:00
20:42:00
20:43:00
20:44:00
20:45:00
15/5/2026, 20:47:25
50th percentile: 160
95th percentile: 560
Number of Users
Number of Users
12
10
8
6
4
2
0
20:37:00
20:38:00
20:39:00
20:40:00
20:41:00
20:42:00
20:43:00
20:44:00
20:45:00
20:46:00
20:47:00
15/5/2026, 20:47:25
Number of Users: 9

¡Gracias por su atención!

¿Preguntas?